

Universidad Rafael Landívar.

Facultad de Ingeniería.

ingeniería en informática y Sistemas.

Laboratorio de Pensamiento Computacional, Sección 16

Docente: Ing. José Castillo

ACTIVIDAD DE LABORATORIO No. 13

“Introducción a clases y POO”

Estudiante: Barrios Molina Ingrid Jimenna

Carné:1059026

Guatemala, 2 de mayo de 2026

CONCEPTOS CLAVE

- CLASE

Es un modelo, plantilla o "plano" que define la estructura y el comportamiento de los objetos. Encapsula atributos (datos/características) y métodos (comportamientos/funciones) que compartirán las instancias creadas a partir de ella, como moldes para crear objetos reales.

- OBJETO

Es una unidad funcional que combina estado y comportamiento en una sola entidad. Representa una instancia concreta de una clase, simulando objetos del mundo real (ej. un usuario, un producto) con atributos y métodos propios.

- ATRIBUTO

Son características, propiedades o datos que definen el estado de un objeto. Representan información específica como nombre, color o tamaño, y se guardan en variables miembro dentro de una clase, permitiendo diferenciar un objeto de otro.

- MÉTODO

En programación orientada a objeto, los métodos son funciones o procedimientos definidos dentro de una clase que representan el comportamiento, acciones o capacidades de un objeto. Permiten manipular los atributos del objeto, reutilizar código y enviar mensajes entre objetos, garantizando la encapsulación.

- CONSTRUCCIÓN DE CLASE

Es el proceso de crear una plantilla que define las características y comportamientos de una entidad del mundo real. Una clase agrupa datos (atributos) y funcionalidades (métodos) en una sola unidad, lo que permite crear objetos inteligentes e interactivos.

- PROGRAMACIÓN ORIENTADA A OBJETOS

Es un paradigma de diseño de software que organiza el código en torno a "objetos" en lugar de funciones. Modela elementos del mundo real mediante clases y objetos, promoviendo la reutilización, encapsulación y el mantenimiento del código. Se basa en cuatro pilares: encapsulación, abstracción, herencia y polimorfismo.

- ABSTRACCIÓN

Es el proceso de identificar y modelar solo las características esenciales de un objeto, ocultando los detalles complejos e innecesarios de su implementación interna. Simplifica sistemas complejos enfocándose en el qué hace un objeto en lugar de cómo lo hace, facilitando el mantenimiento y reutilización del código.

ASIGNACIÓN DE ATRIBUTOS Y MÉTODOS

- Un sistema de biblioteca universitario que permite a estudiantes la búsqueda y alquiler de libros.

Libro
Autor: string numeroPaginas: int editorial: string tamañoLetra: int titulo: string
+CambiarDisponibilidad(): void

-autor: string -numeroPaginas: int -editorial: string -tamañoLetra: int -titulo: string
+CambiarDisponibilidad(): void

- Un sistema de hotel que permita a los internautas la búsqueda y reserva de una habitación para su hospedaje.

Habitación Hotel
numeroHabitación: int ultimaLimpieza: datetime precioPorNoche: double disponibilidad: bool consumoMinibar: double
+SolicitarLimpieza(): void +Reservar(): void +Liberar(): void

-numeroHabitación: int -ultimaLimpieza: datetime -precioPorNoche: double -disponibilidad: bool -consumoMinibar: double
+SolicitarLimpieza(): void +Reservar(): void +Liberar(): void

- Un sistema de parqueo que registre el ingreso de los vehículos y calcule el costo de un ticket en función del tiempo de estadía.

Vehiculo
placa: string marca: string color: string
+BajarVentana(): void

-placa: string -marca: string -horaEntrada: datetime -horaSalida: datetime -color: string
+ValidarTicket(): void +BajarVentana(): void

Estacionamiento
horaEntrada: datetime hora salida: datetime
+ValidarTicket(): void

- Un portal académico universitario que gestione a los estudiantes y cursos impartidos. Este manejará toda la logística y gastos administrativos relacionados con ambos.

Estudiante
carnet: int
correoElectronico: string
carrera: string
PagarMatricula(): void

-carnet: int
-calificaciónFinal: double
-cupoMaximo: int
-carnetDocente: int
-correoElectronico: string
-carrera: string
+PagarMatricula(): void
+AgregarEstudiante(): void

Curso
calificaciónFinal: double
cupoMaximo: int
carnetDocente: int
AgregarEstudiante(): void

CREACIÓN CONCEPTUAL DE CLASES

- Una compañía con varios zoológicos a lo largo de Centroamérica requiere un programa de asistencia para la alimentación de animales. El sistema permitirá seleccionar un zoológico (Existe uno en cada país de la región) y escoger un animal del listado. Deberá mostrar las características físicas del animal, su clasificación alimenticia (carnívoro, herbívoro, etc.), su dieta y su última fecha de alimentación.

Zoológico
nombre: string
pais: string
capacidadAnimales: int
+SeleccionarZoo(): void
+ListarAnimales(): void

nombre: string
pais: string
capacidadAnimales: int
especie: int
caracteristicasFísicas: string
clasificaciónAlimenticia:string
dieta:string
ultimaAlimentación: datetime
+SeleccionarZoo(): void
+ListarAnimales(): void
+MostrarFicha(): void
+ActualizarUltimaAlimentación(): void

Animal
especie: int
caracteristicasFísicas: string
clasificaciónAlimenticia:string
dieta:string
ultimaAlimentación: datetime
+MostrarFicha(): void
+ActualizarUltimaAlimentación(): void

- Se requiere diseñar una nueva aplicación para una cadena de cines popular. El programa debe permitir a los usuarios consultar los horarios de las películas, número de asientos disponibles en la sala, información general de la película (sinopsis, avance, etc) y todo aquello que incentive a comprar un boleto.

Películas
titulo: string sinopsis: string avance: string clasificacion: string.
+ReproducirTrailer(): void

titulo: string sinopsis: string avance: string clasificacion: string numeroSala: int horarios: datetime asientosDisponibles: int tipoProyeccion: string
+ReproducirTrailer(): void +ConsultarDisponibilidad(): int +VenderBoleto(): void

Sala de Cine
numeroSala: int horarios: datetime asientosDisponibles: int tipoProyeccion: string
+ConsultarDisponibilidad(): int +VenderBoleto(): void

- Un gimnasio requiere un sistema para gestionar a sus clientes afiliados. El modelo de negocio ofrece dos tipos de membresías:
- Regular: Con un precio de Q175.00, los afiliados solamente pueden asistir a una sede.
- Premium: Con un precio de Q215.00, los afiliados pueden entrenar en la sede de su preferencia. Las sedes están ubicadas en múltiples puntos de la ciudad de Guatemala. Se espera llevar un seguimiento de la afluencia de afiliados en cada sede y si sobrepasa su capacidad máxima.

Sede Gimnasio
nombreSede: string ubicacion: string capacidadMaxima: int afluenciaActual: int
+RegistrarIngreso(): void, +RegistrarSalida(): void, +VerificarCapacidad(): bool

nombreSede: string ubicacion: string capacidadMaxima: int afluenciaActual: int nombre: string tipoMembresia: string cuotaMensual: double
+RegistrarIngreso(): void +RegistrarSalida(): void +VerificarCapacidad(): bool +PagarCuota(): void +CambiarSedeAsignada(): void

Afiliado
nombre: string tipoMembresia: string cuotaMensual: double
+PagarCuota(): void +CambiarSedeAsignada(): void

Se desea crear el sistema que gestione un torneo eliminatorio de futbol. El sistema deberá gestionar la información de los equipos e integrantes inscritos en el torneo. Debe considerar los siguientes requerimientos:

- Por regulación cada miembro debe cancelar una cuota de inscripción, de lo contrario no podrá participar en el torneo.
- Cada equipo deberá contar con un mínimo de 8 jugadores y un máximo de 12 para poder participar en el torneo.
- Cada jugador se registrará con un número y posición única. Si este recibe 1 tarjeta roja será eliminado del torneo. o Un equipo puede ser descalificado si no se presenta a 3 partidos.
- Cada equipo debe registrar un jugador como capitán, En caso de una lesión deben designar un nuevo capitán.
- Al final del torneo se entregarán dos premios; uno al equipo ganador y otra al equipo que haya anotado más goles durante el torneo. En caso de un empate se utilizará la diferencia de goles (goles a favor - goles en contra).

Equipo de futbol
nombreEquipo: string listaJugadores: List<Jugador> partidosNoPresentados: int golesFavor: int golesContra: int
+CalcularDiferenciaGoles(): int +VerificarInscripcion(): bool +DesignarCapitan(): void

nombreEquipo: string listaJugadores: List<Jugador> partidosNoPresentados: int golesFavor: int golesContra: int nombre: string numeroCamisola: int posicion: string cuotaCancelada: bool tarjetas Rojas: int esCapitan: bool
+CalcularDiferenciaGoles(): int +VerificarInscripcion(): bool +DesignarCapitan(): void +RegistrarGol(): void +RecibirTarjeta(): void

Jugador
nombre: string numeroCamisola: int posicion: string cuotaCancelada: bool tarjetas Rojas: int esCapitan: bool
+RegistrarGol(): void +RecibirTarjeta(): void